

Archon AI

Product Requirements Document (PRD)

Version

v1.0

Author

Lakshay

Project Type

Retrieval-Augmented Generation (RAG) System

Project Goal

Build an AI-powered repository assistant capable of understanding, searching, retrieving, and explaining codebases using semantic search, metadata-aware retrieval, and Large Language Models.

Archon AI should allow developers to ask natural language questions about a GitHub repository and receive accurate, context-aware answers grounded in the repository's actual code.

1. Problem Statement

Modern software repositories contain hundreds or thousands of files.

Developers often struggle to:

- Understand unfamiliar codebases
- Locate important functions
- Trace execution flows
- Find configurations
- Understand architecture decisions
- Onboard to new projects

Traditional search methods rely on exact keyword matching and often fail when users do not know the exact function names or file locations.

Archon AI solves this problem by transforming repositories into a searchable knowledge base using Retrieval-Augmented Generation.

2. Product Vision

Archon AI acts as an intelligent repository expert.

Instead of manually navigating files, developers can ask questions such as:

- How does training start?
- Where is LoRA configured?
- How is authentication implemented?
- Explain the architecture of this repository.
- Which file loads the dataset?
- What is the execution flow of this application?

Archon AI retrieves relevant code and generates grounded explanations.

3. Target Users

Primary Users

- Software Developers
- AI Engineers
- Machine Learning Engineers
- Open Source Contributors

Secondary Users

- Students learning large codebases
 - Technical Interview Candidates
 - Engineering Teams
-

4. Success Criteria

Archon AI should:

- Index an entire repository
 - Retrieve relevant code sections
 - Explain code in natural language
 - Show source files used for retrieval
 - Reduce repository exploration time
-

5. Core Features

Repository Ingestion

Input:

GitHub Repository URL

Example:

<https://github.com/user/project>

System will:

- Clone repository
 - Scan files
 - Filter unsupported files
 - Build internal dataset
-

Semantic Code Chunking

Supported entities:

- Functions
- Classes
- Methods
- Configuration Files
- Documentation Files

Each entity becomes an independent retrieval unit.

Metadata Generation

Every chunk stores metadata.

Example:

File Path

Function Name

Language

Chunk Type

Repository Name

This metadata improves retrieval quality.

Embedding Generation

Each chunk is converted into a vector representation.

Purpose:

Enable semantic search.

Vector Storage

Store embeddings in ChromaDB.

Vector database serves as the retrieval layer.

Natural Language Question Answering

Users can ask questions in plain English.

Example:

Where is model training initialized?

System retrieves relevant chunks and generates an answer.

Source Attribution

Every response should display:

- Source File
- Function/Class Name
- Retrieval Score

This improves trust and explainability.

6. Functional Requirements

Repository Loader

Input: GitHub URL

Output: Local repository copy

File Parser

Supported Formats:

- .py
- .md
- .yaml
- .yml
- .json
- .txt

Ignored:

- Images
 - Executables
 - Virtual Environments
 - Build Artifacts
-

Chunk Generator

Transform repository files into semantic chunks.

Examples:

Function Chunk

Class Chunk

Documentation Chunk

Configuration Chunk

Embedding Pipeline

Input: Chunk

Output: Embedding Vector

Model:

all-MiniLM-L6-v2

Vector Index

Store:

- Chunk Content
- Metadata
- Embeddings

Database:

ChromaDB

Retriever

Input: User Question

Output: Top-K Relevant Chunks

Default:

Top 5 Chunks

Prompt Builder

Combine:

Question

Retrieved Context

Repository Metadata

Create final prompt.

LLM Response Generator

Input:

Prompt

Output:

Grounded Explanation

7. System Architecture

Repository ↓ Repository Loader ↓ File Parser ↓ Semantic Chunker ↓ Metadata Generator ↓ Embedding Model ↓ ChromaDB ↓

User Question ↓ Embedding ↓ Retriever ↓ Top-K Chunks ↓ Prompt Builder ↓ LLM ↓ Answer + Sources

8. Technical Stack

Language

Python

Repository Access

GitPython

Parsing

AST Parser

Pathlib

OS

Embedding Model

Sentence Transformers

all-MiniLM-L6-v2

Vector Database

ChromaDB

Retrieval

Cosine Similarity

Top-K Search

LLM

TinyLlama

or

Qwen 2.5

via Ollama

Frontend

Streamlit

9. Dataset Design

Each chunk should follow:

```
{ "id": "...", "repo": "...", "path": "...", "language": "...", "chunk_type": "...", "symbol": "...", "content": "...",  
  "summary": "... " }
```


This becomes the canonical repository dataset.

10. Project Structure

```
archon-ai/  
  
├── data/ | ├── repositories/ | └── processed/ | ├── chroma_db/ | ├── src/ | ├──  
ingestion/ | | ├── clone_repo.py | | ├── parser.py | | └── file_filter.py | | | ├── chunking/  
| | ├── ast_chunker.py | | └── metadata_builder.py | | | ├── embeddings/ | | └──  
embedder.py | | | ├── vectorstore/ | | └── chroma_manager.py | | | ├── retrieval/ | |  
└── retriever.py | | | ├── prompting/ | | └── prompt_builder.py | | | ├── llm/ | | └──  
generator.py | | | └── pipeline/ | └── rag_pipeline.py | ├── app/ | └── streamlit_app.py |  
└── tests/ | ├── requirements.txt | ├── README.md | └── main.py
```

11. Development Roadmap

- Phase 1
 - Repository Ingestion
 - File Parsing
 - Dataset Generation

- Phase 2
 - Embeddings
 - ChromaDB
 - Basic Retrieval

- Phase 3
 - LLM Integration
 - Question Answering
 - Source Display

Phase 4

Hybrid Search

BM25 + Vector Search

Phase 5

Cross-Encoder Re-ranking

Top-20 → Best-5

Phase 6

Repository Architecture Summaries

Execution Flow Understanding

Advanced Code Reasoning

12. Expected Learning Outcomes

By completing Archon AI, the developer will gain practical experience in:

- End-to-end RAG systems
- Repository indexing
- Semantic chunking
- Metadata engineering
- Vector databases
- Embedding models
- Hybrid retrieval
- Re-ranking
- LLM integration
- Production AI system design

Archon AI serves as a bridge between academic RAG concepts and production-grade retrieval systems.